

Class 23: Requirements Engineering for ML Systems

Session 23 of 36 – Theory: What to Build and Why

Data-driven Systems Engineering

Agenda

- Introduction, user/system requirements, and quality constraints
- Functional, non-functional, and domain requirements
- Specification styles and the requirements document
- Traceability, change, and requirement management

Note

The hands-on practice that used to close this topic now has its own session: Class 25, after a real-world bridge with YesAlps in Class 24.

Why requirements are harder for ML systems

Learning goals

- Distinguish user, system, functional, and non-functional requirements.
- Write testable requirements instead of vague aspirations.
- Translate model behavior into system-level acceptance criteria.
- Connect requirements to later testing, deployment, and monitoring work.

Course Map and Running Cases



Today in the course

Focus: Requirements that make later validation possible.

Bridge: This class connects stakeholder need to technical behavior so that testing, deployment, and monitoring have something concrete to protect.

Fraud case

In the fraud case, requirements include not only prediction quality but latency, logging, explainability, and acceptable false-positive burden.

Pasteurization case

In the pasteurization case, requirements include state visibility, timely alerts, usable thresholds, and operator trust in the dashboard.

Course thread: The course thread here is that weak requirements create confusion that no later engineering step can fully repair.

Why requirements engineering matters so much

- Requirements define what success means before implementation begins.
- Weak requirements create confusion that no amount of modeling skill can repair later.
- In ML systems, requirements must cover the software, the data, the model behavior, and the operating context.

Types of requirement

User requirements

- Natural language
- Service-level descriptions
- Written for customers and operators

System requirements

- Structured and testable
- Detailed behavior and constraints
- Used by developers and reviewers

Why ML systems make requirements harder

- Behavior is partly learned from data rather than fully hand-coded.
- Labels may be delayed, noisy, or disputed.
- Model performance depends on future data conditions, not only current test data.
- Stakeholders may ask for “accuracy” when they really mean trust, fairness, speed, or interpretability.

Bad requirement vs good requirement

Weak

The fraud system should be fast, accurate, and easy to use.

Better

For a batch of 100 review requests, the API shall return a response within 800 ms for at least 95% of requests, and the review dashboard shall display the top 3 risk indicators used in scoring.

Functional, non-functional, and domain requirements

- Functional: what services the system provides.
- Non-functional: timing, reliability, security, usability, maintainability.
- Domain: constraints imposed by the operating environment.

Example

In pasteurization monitoring, "detect abnormal HeatUp behavior" is functional; "alert within 3 seconds" is non-functional; "respect sensor calibration ranges" is domain-specific.

Qualities of a good requirement

- Necessary: it supports a real stakeholder need.
- Testable: someone can verify whether it is satisfied.
- Unambiguous: it is unlikely to be interpreted in different ways.
- Feasible: it fits known technical and organizational constraints.
- Traceable: it can be linked to design, tests, and later changes.

Examples tailored to this repository

- Functional: the Flask service shall expose 'POST /predict' and return JSON.
- Functional: the pasteurization dashboard shall refresh live sensor plots from the stream endpoint.
- Non-functional: model inference shall remain deterministic for a fixed artifact and input.
- Non-functional: drift alerts shall be logged with timestamp and monitored variable.

User stories are not enough by themselves

- User stories are helpful for conversation, prioritization, and scope.
- They still need acceptance criteria to become testable engineering targets.
- A story without measurable acceptance can hide major ambiguity.

Example

“As an analyst, I want a fraud score so that I can review risky transactions” is useful, but it still leaves latency, explanation, threshold logic, and failure behavior unspecified.

The software requirements document

- States what the system should do, not how it should be implemented.
- Should support communication, traceability, validation, and change control.
- Varies with context: student prototypes need less detail than regulated industrial systems.

Traceability links requirements to later work

- A requirement should connect to design choices, tests, deployment gates, and monitoring rules.
- Traceability makes change management easier because teams can see what else is affected when one requirement changes.
- It is especially valuable when the model, pipeline, and interface evolve at different speeds.

Ways to write specifications

- Natural language
- Structured natural language
- Form-based requirement templates
- Graphical models such as use cases and sequence diagrams
- Lightweight formal or tabular specifications when risk is high

Requirements management continues after release

- Requirements are not frozen forever because systems, users, and regulations change.
- Management includes versioning, prioritization, traceability, and impact analysis.
- In ML systems, drift and changing labels can indirectly trigger new requirements.

Why requirements change

- Business priorities shift.
- New data sources or interfaces appear.
- Regulations or privacy constraints tighten.
- Real users expose missing assumptions.
- The model itself changes what stakeholders ask for.

Requirements connect this course together

- Modeling needs evaluation criteria.
- Delivery pipelines need release checks and acceptance gates.
- Monitoring needs rules about what deviations matter and who should be informed.
- MLOps later depends on requirements that were explicit enough to automate.

Question

“The model should be accurate” is a common ask from stakeholders. Is it a requirement?

- This forces the distinction between an aspiration and a testable statement.
- It sets up Class 25, where teams turn this kind of ask into something they could actually verify.

Papers and materials

IEEE recommended practice for software requirements specifications; Sommerville, Software Engineering; Practical MLOps, requirements-to-production perspective; Practical overview of functional and non-functional requirements; User stories and acceptance criteria guide.

From This Class to the Next

What stays with us

- Requirements make modeling, delivery, and monitoring choices accountable to stakeholder goals.
- Functional and non-functional expectations are both essential in ML systems.
- Traceable requirements help teams decide what to build, what to measure, and what to automate.

Project use

A project proposal becomes stronger when it can connect dataset choices, model metrics, interface behavior, and operational checks back to concrete requirements.

Next connection

Class 24 brings in YesAlps, an external partner, to ground requirements and costs in a real engagement before Class 25 puts this theory into practice.

Course thread: Explicit expectations make it possible to choose a realistic development process, which is exactly where the course goes next.